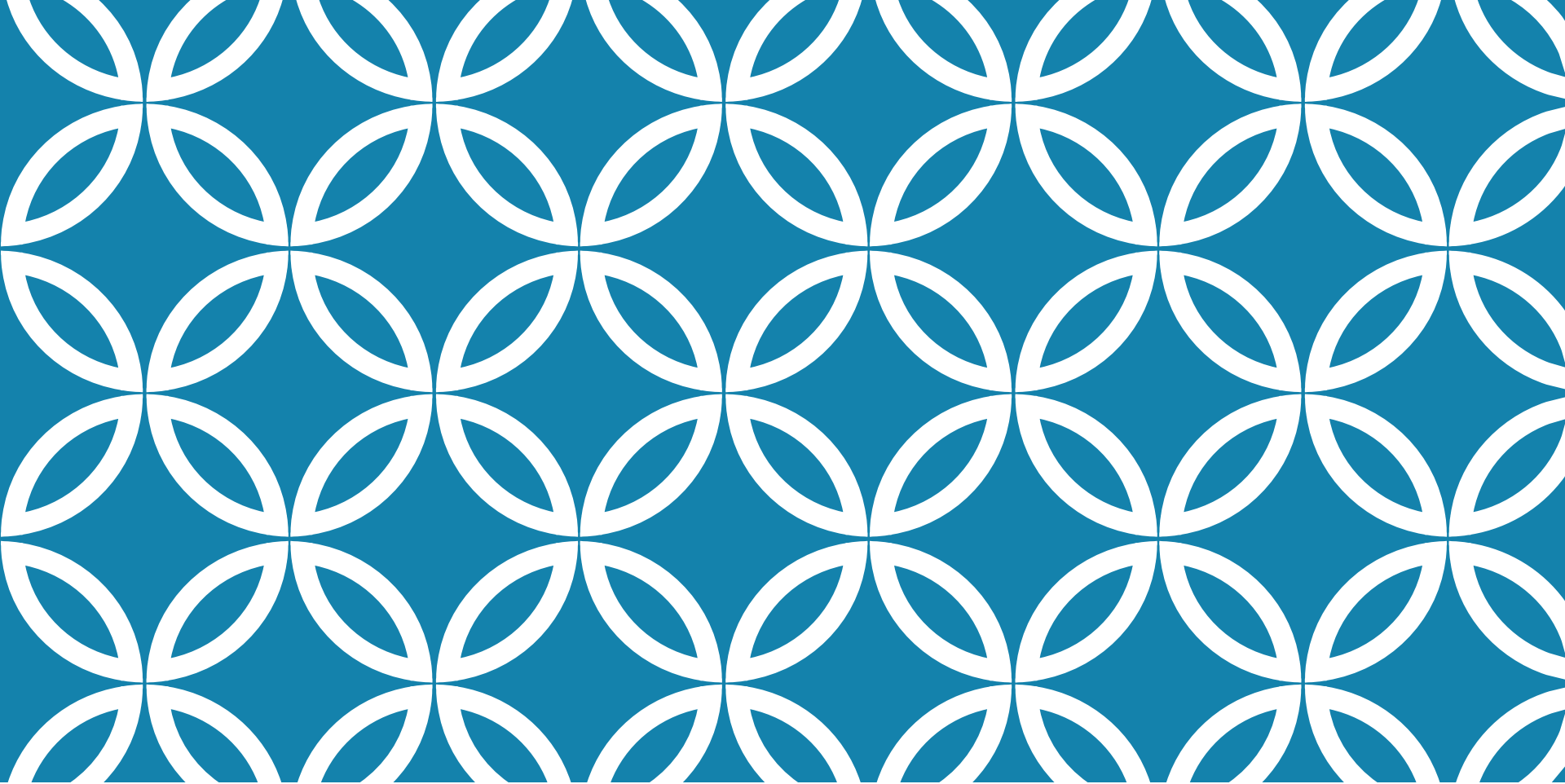




IT SOCIETY

INTRODUCTION

DATABASES

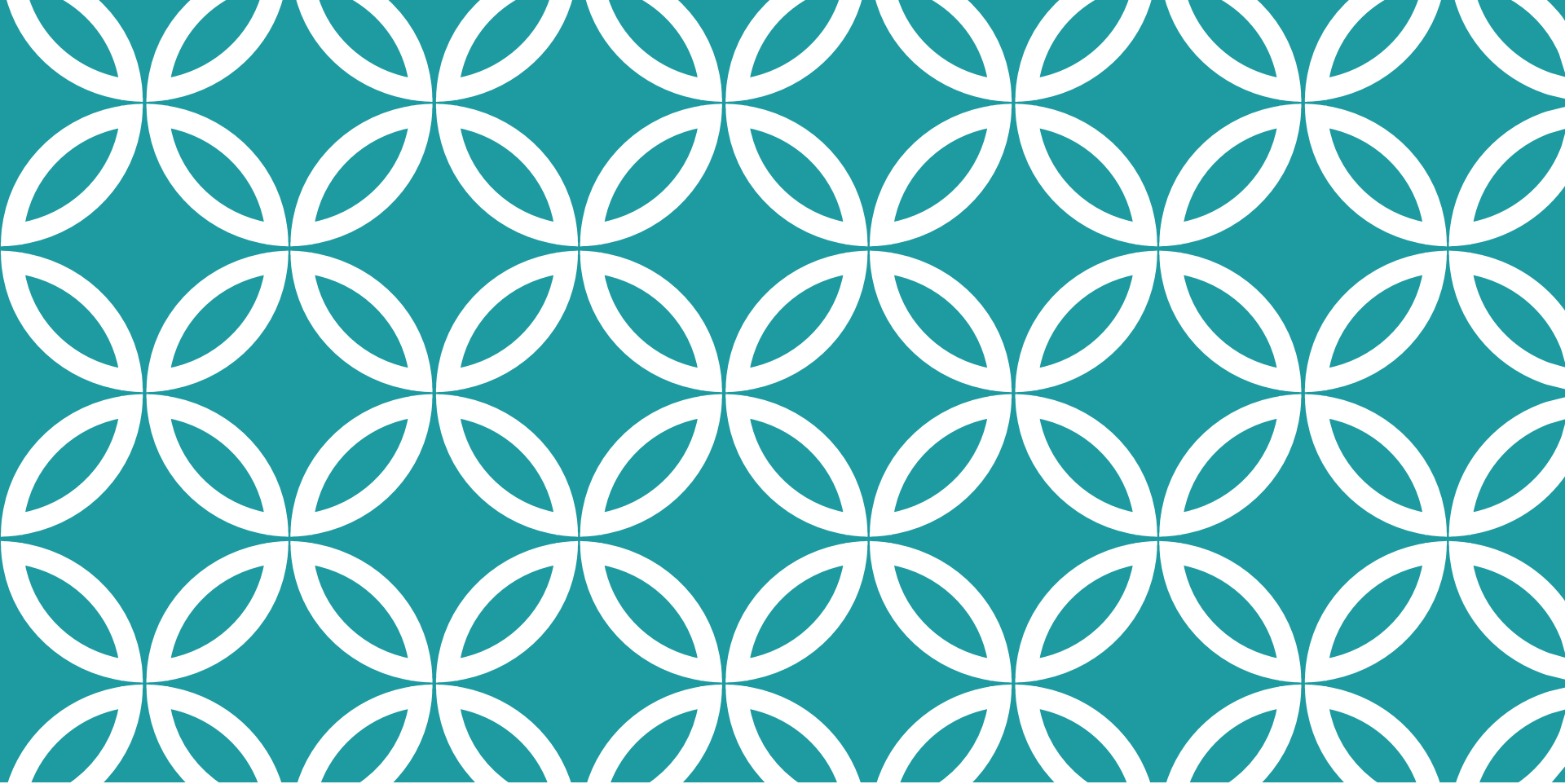
TABLE OF CONTENT

<u>Introduction</u>	5
<u>Types of JOINS</u>	6
<u>INNER JOIN</u>	7
<u>LEFT JOIN</u>	11
<u>RIGHT JOIN</u>	13
<u>CROSS JOIN</u>	15
<u>CONCLUSION</u>	19



RELATIONAL DATABASES





JOINS

**INNER JOIN,
LEFT JOIN,
RIGHT JOIN &
CROSS JOIN**

INTRODUCTION

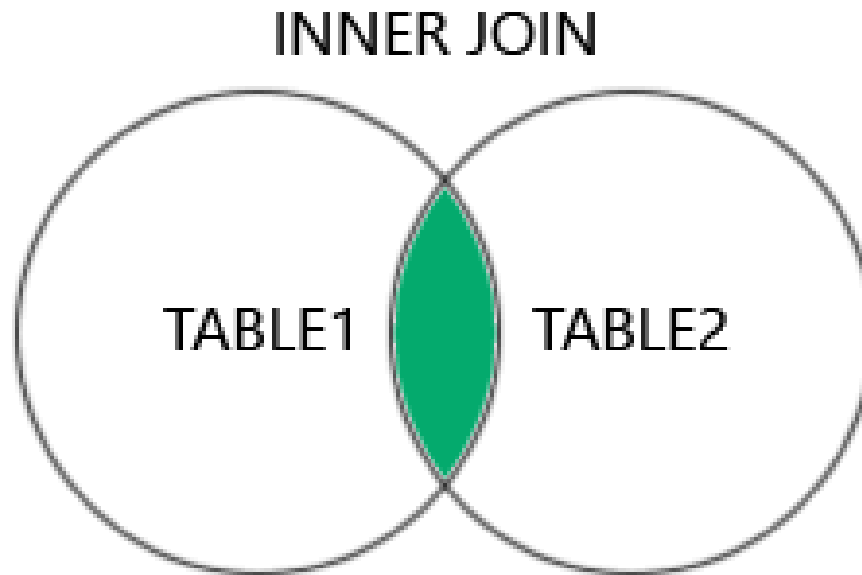
- The **JOIN** clause is used to combine rows from two or more tables, based on a related column between them.
- The most commonly used type of JOINS include;
 1. INNER JOIN
 2. LEFT JOIN
 3. RIGHT JOIN
 4. CROSS JOIN

TYPES OF JOINS

TYPE	DESCRIPTION
INNER JOIN	Matches rows that exist in both tables.
LEFT JOIN	All rows from left table + matches from right.
RIGHT JOIN	All rows from right table + matches from left.
FULL OUTER JOIN	All rows from both tables. NOT SUPPORTED!
CROSS JOIN	Cartesian product of both tables. Every combination of rows (table1 rows × table2 rows).
SELF JOIN	Table joined with itself.
NATURAL JOIN	Automatically joins on columns with same name.
STRAIGHT_JOIN	Forces left table to be read first.

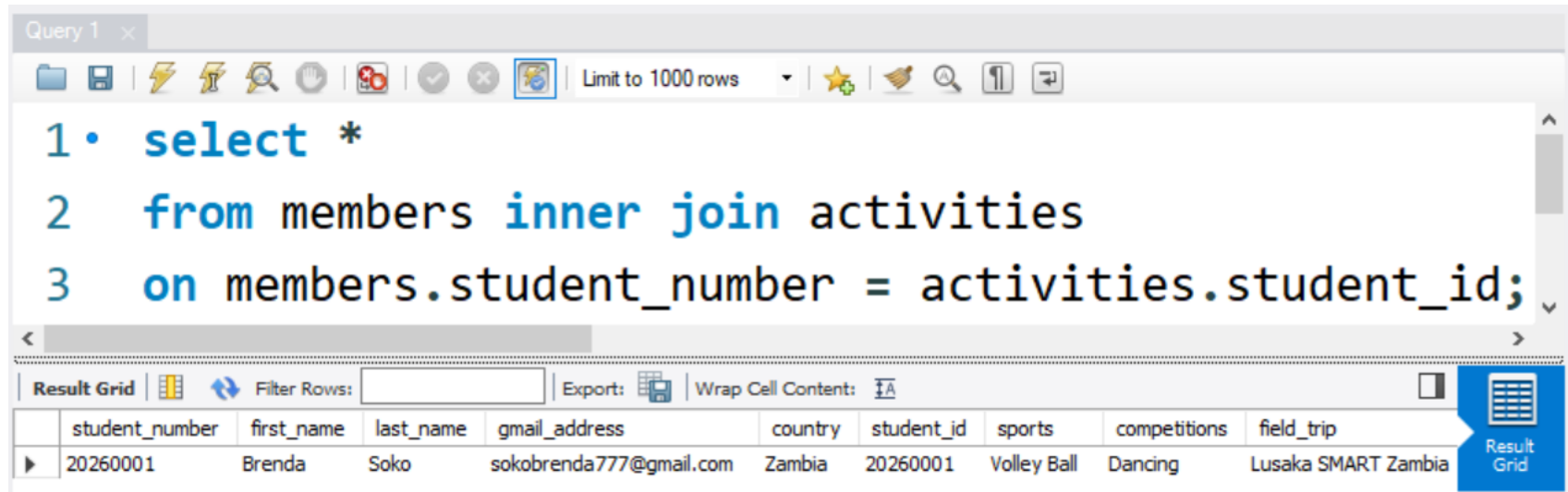
17. INNER JOIN

- The **INNER JOIN** clause returns only rows that have matching values in both tables.



17. INNER JOIN (CONTINUES...)

- Let's take an example below;



The screenshot shows a database query editor window titled "Query 1". The query is as follows:

```
1 • select *
2   from members inner join activities
3   on members.student_number = activities.student_id;
```

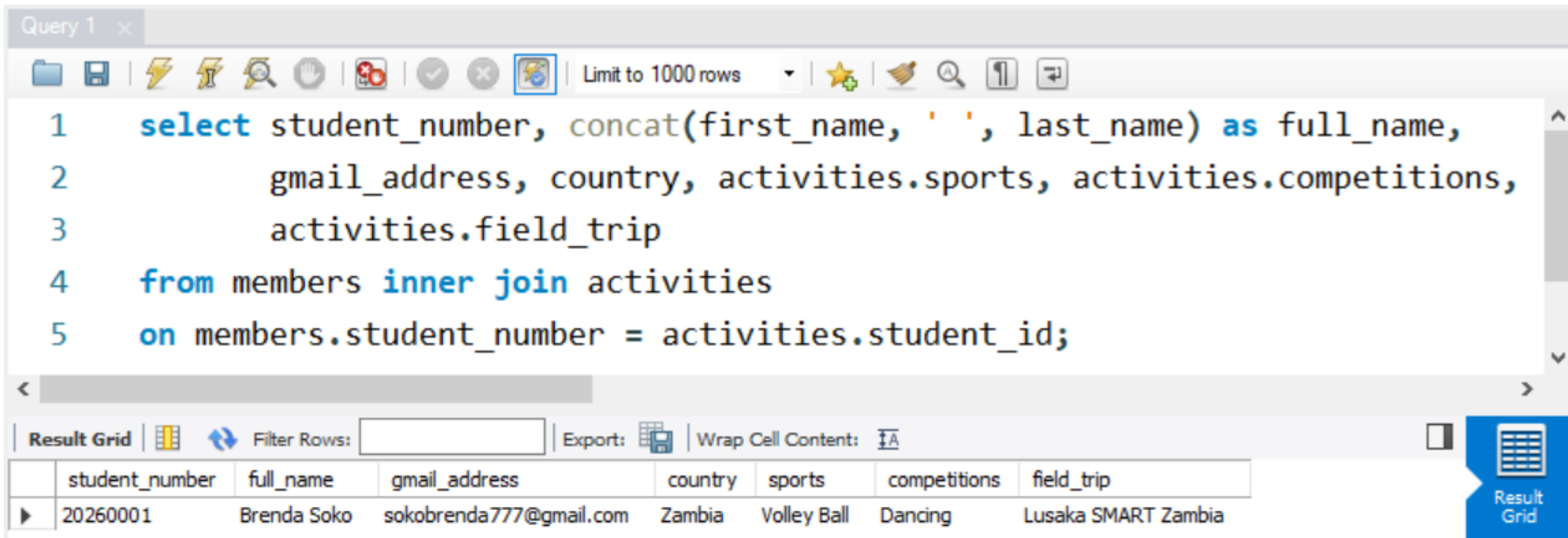
Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has the following columns: student_number, first_name, last_name, gmail_address, country, student_id, sports, competitions, and field_trip. The first row of data is:

student_number	first_name	last_name	gmail_address	country	student_id	sports	competitions	field_trip
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia	20260001	Volley Ball	Dancing	Lusaka SMART Zambia

- **NOTE:** Brenda's records are in both tables.

17. INNER JOIN (CONTINUES...)

- We can also customize the columns that should be displayed.



The screenshot shows a database query editor window titled "Query 1". The query is an SQL statement that performs an inner join between the "members" and "activities" tables. The query is as follows:

```
1 select student_number, concat(first_name, ' ', last_name) as full_name,  
2      gmail_address, country, activities.sports, activities.competitions,  
3      activities.field_trip  
4 from members inner join activities  
5 on members.student_number = activities.student_id;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has 7 columns: student_number, full_name, gmail_address, country, sports, competitions, and field_trip. The first row of data is:

student_number	full_name	gmail_address	country	sports	competitions	field_trip
20260001	Brenda Soko	sokobrenda777@gmail.com	Zambia	Volley Ball	Dancing	Lusaka SMART Zambia

17. INNER JOIN (CONTINUES...)

- INNER JOIN is the same as just using JOIN itself.



The screenshot shows a database query editor window titled "Query 1". The query is as follows:

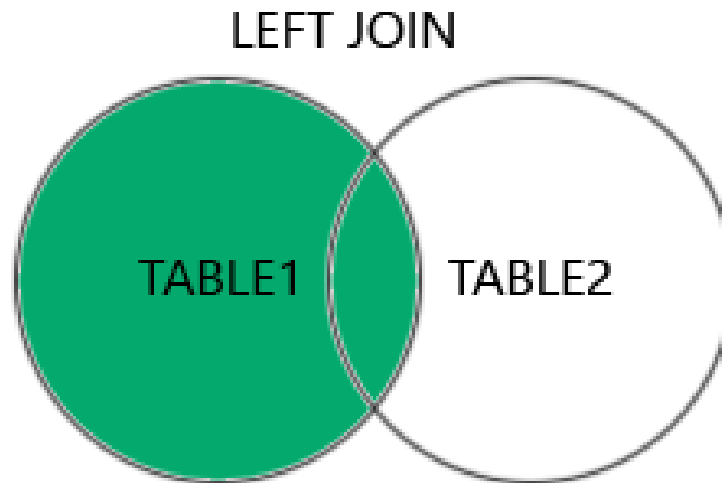
```
1 select student_number, concat(first_name, ' ', last_name) as full_name,  
2     gmail_address, country, activities.sports, activities.competitions,  
3     activities.field_trip  
4 from members join activities  
5 on members.student_number = activities.student_id;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has the following columns: student_number, full_name, gmail_address, country, sports, competitions, and field_trip. The first row of data is:

student_number	full_name	gmail_address	country	sports	competitions	field_trip
20260001	Brenda Soko	sokobrenda777@gmail.com	Zambia	Volley Ball	Dancing	Lusaka SMART Zambia

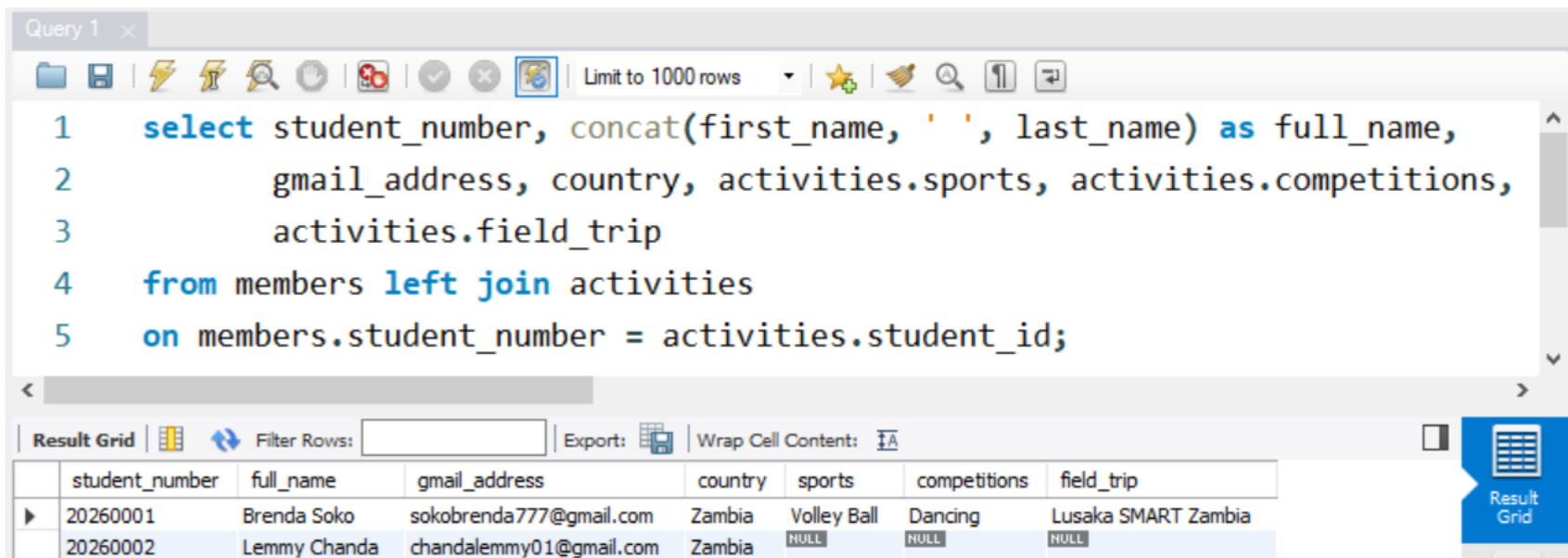
18. LEFT JOIN

- The **LEFT JOIN** clause returns all rows from the left table (table1), and only the matched rows from the right table (table2).



18. LEFT JOIN (CONTINUES...)

- Remember, the members table has two records, following our syntax, we are starting with the members table (left side) so all its data will be displayed.



The screenshot shows a database query editor window titled "Query 1". The query is as follows:

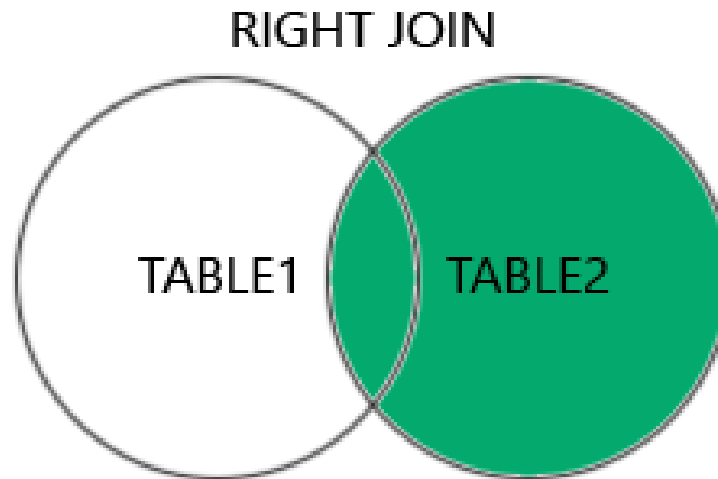
```
1 select student_number, concat(first_name, ' ', last_name) as full_name,  
2     gmail_address, country, activities.sports, activities.competitions,  
3     activities.field_trip  
4 from members left join activities  
5 on members.student_number = activities.student_id;
```

Below the query, the "Result Grid" is displayed, showing the results of the query. The grid has 7 columns: student_number, full_name, gmail_address, country, sports, competitions, and field_trip. There are two rows of data:

student_number	full_name	gmail_address	country	sports	competitions	field_trip
20260001	Brenda Soko	sokobrenda777@gmail.com	Zambia	Volley Ball	Dancing	Lusaka SMART Zambia
20260002	Lemmy Chanda	chandalemmy01@gmail.com	Zambia	NULL	NULL	NULL

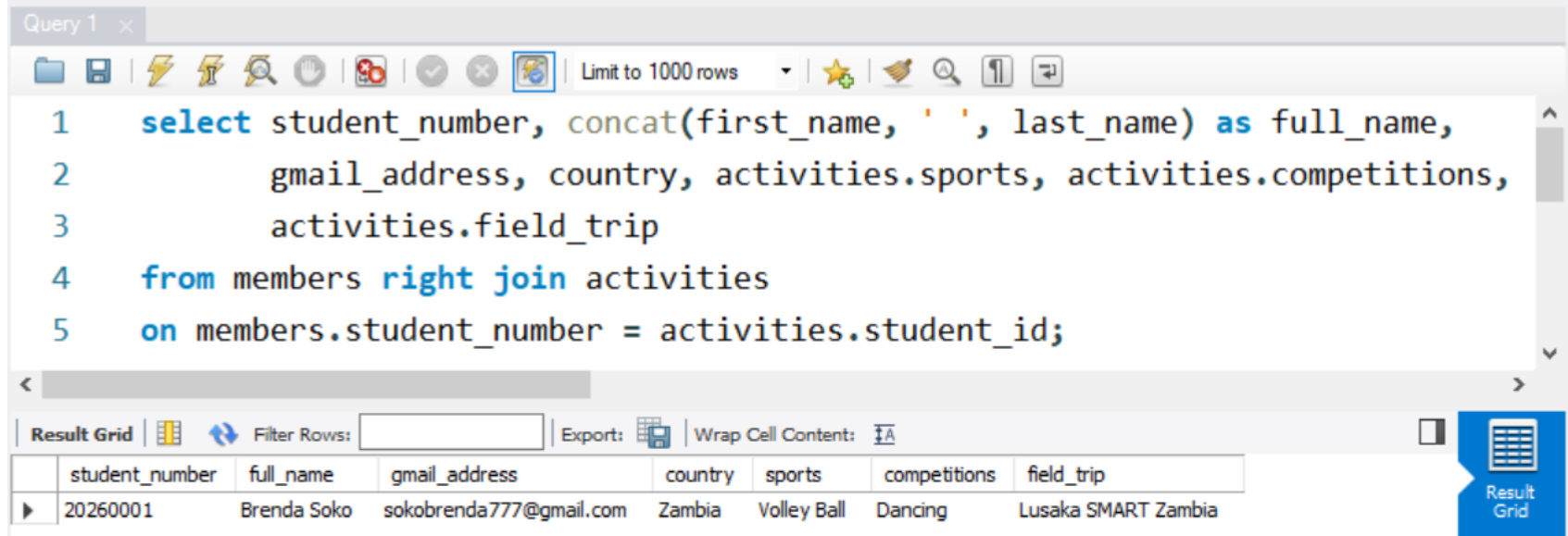
19. RIGHT JOIN

- The **RIGHT JOIN** clause returns all rows from the right table (table2), and only the matched rows from the left table (table1).



19. RIGHT JOIN (CONTINUES...)

- Based on our syntax again, the activities table is on the right side and it has only one record, so that will be our result.



The screenshot shows a database query editor window titled "Query 1". The query is as follows:

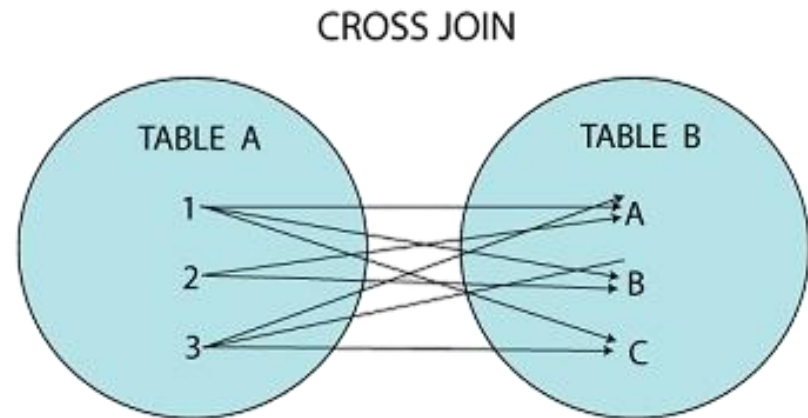
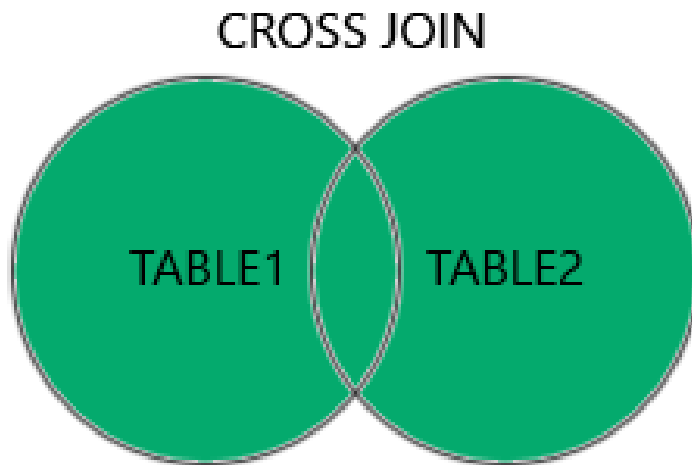
```
1 select student_number, concat(first_name, ' ', last_name) as full_name,  
2     gmail_address, country, activities.sports, activities.competitions,  
3     activities.field_trip  
4 from members right join activities  
5 on members.student_number = activities.student_id;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has 8 columns: student_number, full_name, gmail_address, country, sports, competitions, and field_trip. The first row of data is:

student_number	full_name	gmail_address	country	sports	competitions	field_trip
20260001	Brenda Soko	sokobrenda777@gmail.com	Zambia	Volley Ball	Dancing	Lusaka SMART Zambia

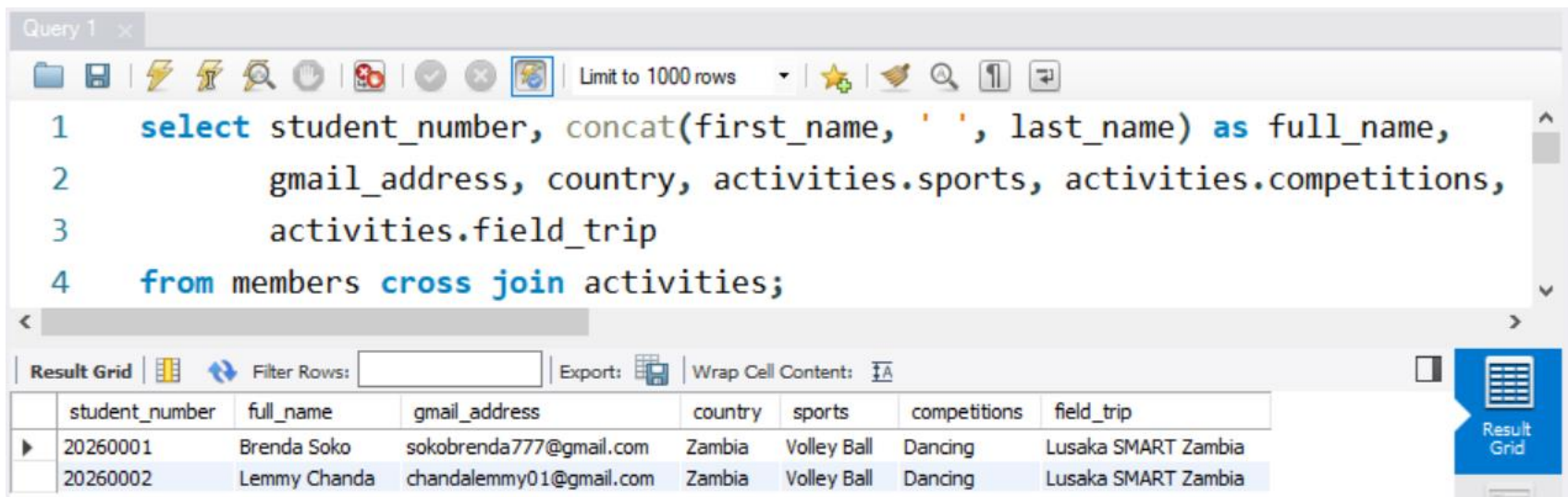
20. CROSS JOIN

- The **CROSS JOIN** clause returns the Cartesian product of two or more tables (combines every row from the first table with every row from the second table).



20. CROSS JOIN (CONTINUES...)

- Okay, this will look a bit complicated but that's a CROSS JOIN 🤖



The screenshot shows a database query editor window titled "Query 1". The SQL query is as follows:

```
1 select student_number, concat(first_name, ' ', last_name) as full_name,  
2      gmail_address, country, activities.sports, activities.competitions,  
3      activities.field_trip  
4 from members cross join activities;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has 8 columns: student_number, full_name, gmail_address, country, sports, competitions, and field_trip. There are 2 rows of data.

student_number	full_name	gmail_address	country	sports	competitions	field_trip
20260001	Brenda Soko	sokobrenda777@gmail.com	Zambia	Volley Ball	Dancing	Lusaka SMART Zambia
20260002	Lemmy Chanda	chandalemmmy01@gmail.com	Zambia	Volley Ball	Dancing	Lusaka SMART Zambia

20. CROSS JOIN (CONTINUES...)

- A **CROSS JOIN** does not use the **ON** clause.
- Since **CROSS JOIN** produces an exhaustive list of combinations, it is used in specific scenarios where full permutations are needed:
- For mapping every student to a certain sport.

NOTICE

Will cover these other types of JOINS once we advance in our database sessions. For now, let's just know these mostly commonly used types.

CONCLUSION

- At this point, we now have an understanding about some types of JOINS in **MySQL**.
- In this third handout, our focus was to understand the basic use of **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, and **CROSS JOIN**.



UP NEXT... UNION